



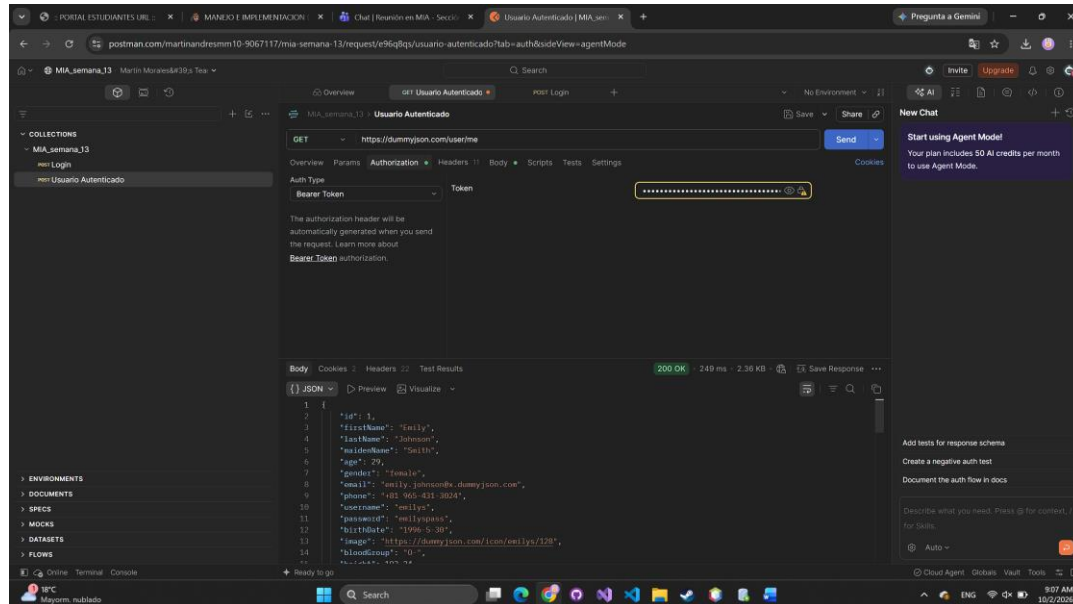
c) Utilice: <https://dummyjson.com/user/me>

d) En Authorization seleccione Bearer Token.

e) Copie el accessToken obtenido en la solicitud Login.

f) Ejecute la solicitud.

g) Verifique que la API devuelve la información del usuario autenticado.



3. Comprobar qué ocurre sin autenticación

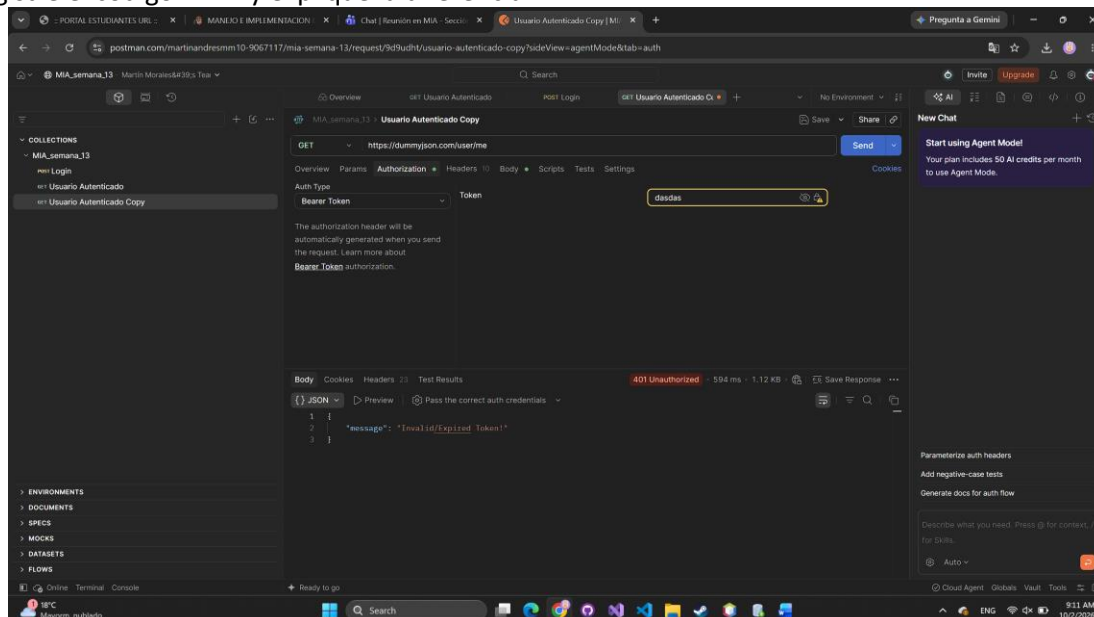
a) Duplique la solicitud Usuario autenticado.

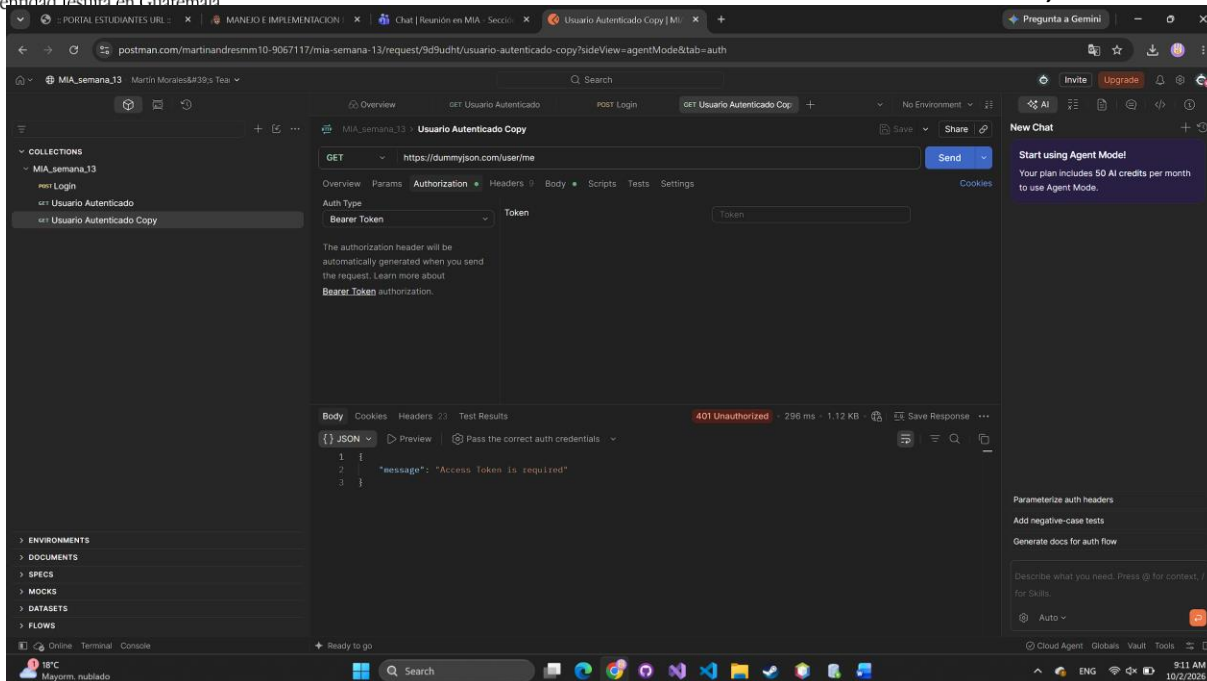
b) Quite el token.

c) Ejecute nuevamente la solicitud.

d) Compare la respuesta con la solicitud autenticada.

e) Registre el código HTTP y explique la diferencia.





4. Evidencia

- Captura de Login mostrando la respuesta.
- Captura de la solicitud autenticada.
- Captura de la prueba sin token.
- Indique qué función cumple el Bearer Token.

La documentación de DummyJSON muestra el flujo de login, la obtención del accessToken y su envío mediante el encabezado Authorization: Bearer.

Parte 2 – Proteger la Connection String de Azure Storage

Utilice como base el programa C# desarrollado anteriormente para conectarse a Azure Storage. No se requiere crear nuevamente el Storage Account

5. Localizar la Connection String

- a) Abra el proyecto C# de Azure Storage.
- b) En el Explorador de soluciones, abra el archivo donde actualmente está escrita la Connection String. En un proyecto sencillo normalmente será Program.cs.
- c) Busque la variable que contiene la Connection String.
- d) Identifique el valor que comienza con información como DefaultEndpointsProtocol=...

6. Reemplazar la Connection String

Reemplace el valor escrito directamente en el programa por una lectura de una variable de entorno llamada: **AZURE_STORAGE_CONNECTION_STRING**

La idea es que el programa siga utilizando la misma variable para conectarse a Azure, pero ahora el valor será obtenido desde el entorno y no desde el código

7. Crear la variable para ejecutar localmente

- a) En la carpeta principal del proyecto, cree un archivo llamado .env.
- b) Dentro del archivo agregue la variable:



AZURE_STORAGE_CONNECTION_STRING=TU_CONNECTION_STRING

- c) Reemplace SU_CONNECTION_STRING por la Connection String de su Storage Account.
- d) Verifique que el programa lea la variable desde el .env y ejecute nuevamente el programa.
- e) Compruebe que el programa continúa funcionando correctamente.
- f) **No publique el archivo .env en GitHub**, ya que contiene la credencial real.

8. Crear el Secret en Github

- a) Ingrese al repositorio de su proyecto en GitHub.
- b) Seleccione Settings.
- c) En el menú lateral seleccione Secrets and variables.
- d) Seleccione Actions.
- e) Seleccione New repository secret.
- f) En Name escriba: AZURE_STORAGE_CONNECTION_STRING
- g) En Secret coloque la Connection String de Azure Storage.
- h) Seleccione Add secret.

The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows a project structure with folders like 'semana-1' through 'semana-13' and files like 'README.md'. The main editor area shows the 'lab12.cs' file, which contains C# code for connecting to Azure Storage. The code uses the 'Azure.Storage.Blobs' package and checks for the 'AZURE_STORAGE_CONNECTION_STRING' environment variable. The 'Program.cs' file is also visible, showing a menu-driven application. The Output pane at the bottom shows the execution of the program, displaying the connection string and the menu options.

```
using Azure;
using Azure.Storage.Blobs;

const string containerName = "lab-archivos";
const string connectionString = "AZURE_STORAGE_CONNECTION_STRING";

BlobServiceClient blobServiceClient;

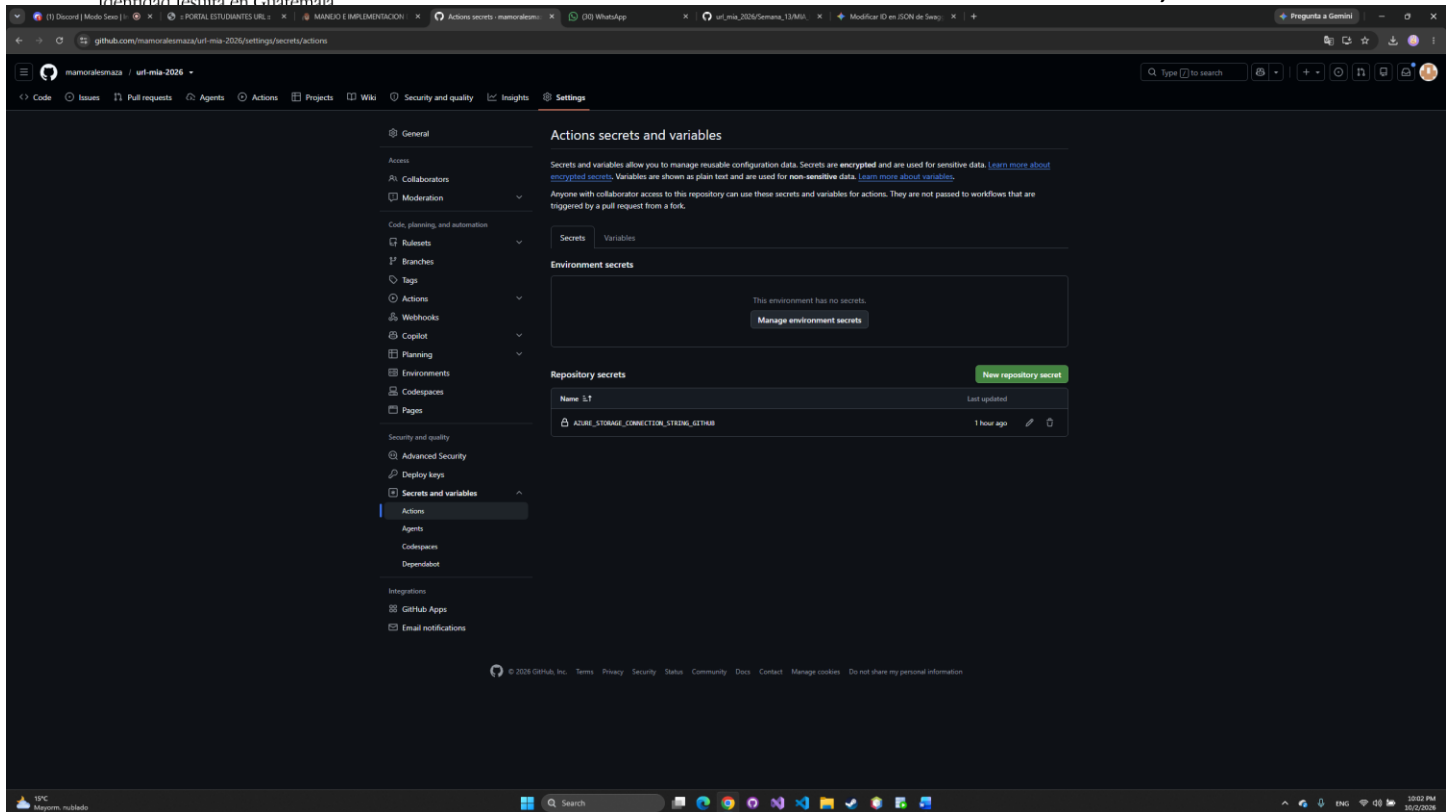
try
{
    blobServiceClient = new BlobServiceClient(connectionString);
}
catch (Exception ex) when (ex is ArgumentException or FormatException)
{
    Console.WriteLine("La configuración de conexión no tiene un formato válido.");
    return;
}

BlobContainerClient containerClient = blobServiceClient.GetBlobContainerClient(containerName);

bool salir = false;
while (!salir)
{
    MostrarMenu();
    string opcion = LeerTexto("Selecciona una opción: ");
}
```

Output:

```
PS C:\Users\martin\OneDrive\Desktop\url-mia-2026> cd semana-12
PS C:\Users\martin\OneDrive\Desktop\url-mia-2026\semana-12> cd lab12
PS C:\Users\martin\OneDrive\Desktop\url-mia-2026\semana-12\lab12> dotnet run
1. Añadir archivo
2. Listar archivos
3. Descargar archivo
4. Eliminar archivo
5. Salir
-----
Selecciona una opción: 4
Nombre exacto del blob: blob
¿El blob existe?
1. Salir archivo
2. Listar archivos
3. Descargar archivo
4. Eliminar archivo
5. Salir
-----
Selecciona una opción: 5
```



Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab2/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.